# Conan: A Hallucination-Resistant, Agentic Retrieval-Augmented Generation Framework for Arabic Legal Reasoning in Egyptian Criminal Law

Mirna Nageh[a,*], Wageh Mostafa[a], Seif Sherif[a], Mariam Mohamed[a], Mayar Mohammed[a]

[a]*Faculty of Computers & Artificial Intelligence, Capital (Helwan) University, Egypt*

**Abstract**

Retrieval-Augmented Generation (RAG) has emerged as the dominant architecture for grounding large language models on domain knowledge, yet its application to Arabic legal information retrieval continues to suffer from hallucinated statutory citations and confabulation when retrieval surfaces topically-related but legally-inadequate context. We present **Conan**, a production-grade Arabic legal assistant for the Egyptian Criminal Code (قانون العقوبات) and the Code of Criminal Procedure (قانون الإجراءات الجنائية). Our contribution is twofold. First, we describe the construction of an Egyptian criminal-law corpus assembled from scratch: 933 legacy-format source files (818 `.doc`, 112 `.pdf`, 3 `.docx`) gathered from authoritative Egyptian legal collections, manually OCR-processed and cleaned into 1,057 UTF-8 documents and indexed as 47,028 chunks. The resulting dataset is released publicly on HuggingFace as *Egyptian_Criminal_Legal_Assistant_RAG_V2*. Second, we introduce a seven-layer grounding-defence pipeline—input gating, citation-grounding prompts with substantive-vs-procedural law disambiguation, evidence validation, corrective retry with a block-list, an article-lookup rescue indexing 1,494 distinct statutory articles, iterative retrieval with adaptive $k$-expansion, and deterministic answer post-processing—layered on top of a hybrid retrieval baseline (FAISS + BM25 + Reciprocal Rank Fusion + cross-encoder reranking). On a 28-question Arabic legal benchmark spanning substantive and procedural questions, the complete system reduces the hallucinated-citation rate from a baseline of 28.6 % to **0.0 %** and the evidence-validation pass rate from 71.4 % to **100 %**. We also report a negative result on synonym-based query expansion. Beyond single-question Q&A, the system escalates open-ended case work to a **six-stage multi-agent agentic pipeline**—document analysis, weakness detection, per-weakness legal-research and precedent retrieval, defence-strategy ranking, and grounded synthesis—in which a deterministic, code-computed timing verdict is injected as a hard fact to eliminate a recurring chronological-reasoning error, every argument is anchored to retrieved authority rather than parametric memory, and a draft → verify → revise self-check closes the argument-level hallucination gap; the pipeline degrades gracefully to a single-shot path on any agent failure. The system is deployed as a production FastAPI service with a five-provider, multi-key failover chain, big-context routing, a zero-token answer cache, an in-process rate gate, a wall-clock cascade budget, persistent multi-turn sessions with compaction, token streaming, automated watch-folder ingestion with hot-reload, and a documented .NET integration contract. Finally, we report a retrieval-quality optimisation phase targeting the retrieval ceiling that bounds

answer correctness—article-aware chunking that keeps each statute article intact in one retrievable unit (live), a widened cross-encoder candidate pool (live), and a BGE-M3 embedding upgrade prepared behind configuration—together with a scored evaluation harness reporting recall@k, article-recall, primary-hit@k, MRR, hallucination rate, abstention, and confidence calibration over a labeled gold set, so every change is measurable. We also report a practical systems finding: on CPU-only hardware the highest-quality embedder can be infeasible to *build* without accelerator access, making the embedder a joint quality/operability choice. We release the corpus, the system, the multi-version evaluation trail, and the evaluation harness.

*Keywords:* Arabic NLP, Retrieval-Augmented Generation, Agentic AI, Multi-Agent Systems, Legal Information Retrieval, Hallucination Mitigation, Egyptian Criminal Law, Article-Lookup Rescue, Iterative Retrieval, Article-Aware Chunking, BGE-M3, Neuro-Symbolic Reasoning, Dataset Construction

## 1. Introduction

The Arabic language is spoken by approximately 422 million people across more than twenty sovereign states, yet it remains comparatively under-served by modern natural language processing (NLP) systems [1]. This under-representation is especially acute in *Arabic legal* NLP, where Modern Standard Arabic (MSA), domain-specific terminology, dialectal variation, and the morphological richness of the language compound the standard challenges of grounded language modelling.

Retrieval-Augmented Generation (RAG) has become the dominant architecture for closing the gap between a frozen LLM's parametric memory and the dynamic, citation-heavy demands of legal practice. By retrieving relevant passages from a curated corpus and conditioning the generator on those passages, RAG can in principle produce answers that are both fluent and traceable [2, 6]. However, recent surveys of Arabic legal RAG—including the Moroccan family-code study of Hrimech et al.[2] and the morphologically-aware retrieval study of Aboasal et al.[5]—identify a recurring failure pattern: the LLM remains liable to *hallucinate statutory citations*, even when grounded retrieval is available, particularly when the user's question maps to articles that were not surfaced in the retrieval top-$k$.

This work makes two complementary contributions. We first describe the *construction of an Egyptian criminal-law corpus from scratch*: 933 legacy-format source files (overwhelmingly `.doc` legacy binaries requiring manual OCR) gathered from authoritative collections of Egyptian legal text and processed into a clean 1,057-document corpus, released publicly on HuggingFace as *Egyptian_Criminal_Legal_Assistant_RAG_V2*. We then present **Conan** (كونان), a hallucination-resistant Arabic legal-AI assistant built on top of this corpus. Conan layers a six-mechanism grounding-defence pipeline on top of a standard hybrid-retrieval architecture (FAISS dense embeddings + BM25 sparse

*Corresponding author.
*Email address:* mirnanagehb.w@gmail.com (Mirna Nageh)

search, fused via Reciprocal Rank Fusion, with cross-encoder reranking). We argue—and our evaluation empirically demonstrates—that *no single technique* is sufficient for hallucination suppression in Arabic legal RAG; a layered approach combining input filtering, prompt engineering, post-hoc validation, article-level metadata lookup, and answer rewriting is required.

### *1.1. Contributions*

1. A publicly-released Egyptian-criminal-law corpus, manually OCR-processed from 818 legacy `.doc` binaries and 112 PDFs into 1,057 UTF-8 documents covering the Penal Code, the Code of Criminal Procedure, cassation rulings, criminal-law treatises, and forensic medicine references (Section 4).
2. A **multi-stage grounding-defence pipeline** (Section 6) reducing the hallucinated-citation rate from 28.6 % to 0.0 % on our benchmark—an absolute reduction of 28.6 percentage points and an 8-fold relative improvement.
3. An **article-lookup rescue subsystem** (Section 6.5) that indexes 1,494 distinct Egyptian-law article numbers from chunk metadata at startup and uses them as a deterministic second-chance grounding source when post-generation evidence validation flags a missing citation.
4. **Iterative retrieval with adaptive $k$-expansion** (Section 6.6) that widens retrieval to $k \in \{14, 21\}$ when validation fails—without an additional LLM call.
5. A detailed **prompt-engineering recipe** (Section 6.2) including substantive-vs-procedural law disambiguation, casual-opener prohibition, fragment refusal, and a block-list retry prompt—together with the leak-detection regex used in post-processing.
6. A **five-version evaluation trail** (Section 9) with per-version CSV outputs, allowing reproducible auditing of every architectural change, plus a *negative result* (Section 11) on heuristic synonym expansion as a cautionary data point for future Arabic RAG work.
7. A document-upload subsystem (Section 8) supporting three retention semantics across `.txt`, `.pdf`, and `.docx` formats.
8. A **procedural-defence reasoning checklist** (Section 6.8) that encodes expert-lawyer case-analysis heuristics (warrant-timeline nullity, identifier mismatch, chain-of-custody, intent-from-profession) into the case-analysis prompts, paired with a *grounding-safe* few-shot exemplar that raises reasoning coverage without reintroducing hallucinated citations.
9. A **six-stage multi-agent agentic case-analysis pipeline** (Section 7) that decomposes open-ended legal work (`/weakness`, `/defense`) into specialised agents—document analysis, weakness detection, per-weakness legal-research and precedent retrieval, defence-strategy ranking, and grounded synthesis—orchestrated with bounded-concurrency `asyncio` fan-out, a deterministic *neuro-symbolic* timing verdict computed in code and injected as a hard fact, retrieval-per-weakness grounding, and an agentic draft → verify → revise self-check, with automatic fallback to a single-shot path on any agent failure.
10. A **production-grade operational architecture** (Sections 5, 10): a five-provider, multi-key failover chain with big-context routing for large case files, a zero-token answer cache, an in-process rate gate and a wall-clock cascade budget that together

bound free-tier exhaustion and worst-case latency, persistent compacting multi-turn sessions, token streaming, automated watch-folder ingestion with in-place index hot-reload, an agentic self-check for defence memoranda (Section 7.3), and a frozen, documented wire contract for an external .NET frontend.

11. A **retrieval-quality optimisation phase with a scored evaluation harness** (Section 9.8): article-aware chunking that keeps each statute article intact in one retrievable unit and a widened cross-encoder candidate pool (both live), plus a BGE-M3 embedding upgrade prepared behind configuration with build/swap tooling—together targeting the retrieval ceiling that bounds answer correctness—accompanied by a labeled-gold-set harness reporting recall@k, article-recall, primary-hit@k, MRR, hallucination rate, abstention, and confidence calibration, so every retrieval/LLM/chunking change is measurable and reproducible. We additionally report the CPU-embedding-cost constraint that defers the embedding rollout to GPU-equipped hardware.

## 2. Related Work

### *2.1. Arabic RAG foundations*

El-Beltagy and Abdallah [1] present a foundational case study of Arabic RAG, evaluating multiple semantic-embedding models and LLMs in the retrieval and generation stages respectively, and explicitly investigating the impact of dialectal variation between document language and query language. Their work establishes the baseline architectural template that the present paper inherits—a semantic retriever feeding a generator LLM—while flagging the central challenge of selecting an embedding model that captures the semantic nuances of Arabic without over-relying on English-dominated pre-training. Alghamdi et al.[4] focus on the retriever component, demonstrating that retrieval quality is the dominant factor in downstream answer correctness—a finding our own iterative-retrieval mechanism (Section 6.6) extends by treating retrieval $k$ as an adaptively-tunable parameter rather than a fixed hyperparameter.

### *2.2. Arabic legal RAG*

Hrimech et al.[2] present the most directly comparable prior work: a RAG system for the *Moroccan* family code, built on BGE-m3 with a custom dataset of 2,500 Arabic question–answer pairs. They explicitly call out the challenges of *legal-terminology adherence*, *content validity of reproduced clauses*, and the *scarcity of annotated Arabic legal corpora*. The dataset contribution of the present work (Section 4) targets the third concern directly: an Egyptian criminal-law corpus released publicly under an open licence.

Aboasal et al.[5] approach Arabic legal IR from a *morphological-and-semantic* angle, introducing a synthetic benchmark of 500 articles and 1,000 Q&A pairs and evaluating the impact of Farasa-based morphological segmentation on multiple retrievers. Their hybrid Farasa-BM25 + Ada v3 configuration reaches MAP= 0.8304 and nDCG@10= 0.8626—figures that informed our choice to keep BM25 as a complementary signal alongside dense FAISS retrieval (Section 5) rather than abandoning it in favour of pure dense retrieval. We extend their hybrid-retrieval philosophy by adding a post-generation defence layer (Section 6) which they do not address.

### 2.3. Benchmarking Arabic legal reasoning

Abu Shairah et al.[3] introduce **ALARB**, an Arabic Legal Argument Reasoning Benchmark comprising over 13,000 commercial court cases from Saudi Arabia, with each case annotated with facts, the court's reasoning chain, the verdict, and the cited regulatory clauses. ALARB sets the methodological precedent for our evaluation harness: a benchmark whose primary axis is not the linguistic *fluency* of the generated text but its *legal-grounding fidelity*—specifically, whether cited statutory articles can be traced to retrieved or otherwise-available source text. Our 28-question benchmark is far smaller in scale, by design, but inherits the philosophy that *what an LLM cites matters more than how it phrases the citation*.

### 2.4. RAG tooling literature

A pragmatic introduction to RAG architecture is provided by the Mini-RAG documentation [6], which decomposes the pipeline into retrieval, augmentation, and generation. While not a peer-reviewed publication, the document is representative of the implementation-oriented tutorial literature that informed many practical design decisions in our system.

### 2.5. Positioning of the present work

Relative to [1, 2], we shift the locus of grounding from the retrieval stage alone to a *post-generation defence pipeline*: even with state-of-the-art retrieval, an LLM may emit a memory-based citation that retrieval did not provide, and detecting and rescuing this case is at least as important as improving the retriever. Relative to [4, 5], we share the focus on hybrid retrieval but extend the operational regime to include iterative $k$-expansion and article-number-keyed rescue. Relative to [3], we adopt the grounding-fidelity evaluation philosophy but apply it to an Egyptian criminal-law corpus rather than a Saudi commercial-law one. None of the prior works release an open Egyptian-criminal-law corpus comparable in scope to the one introduced here.

## 3. Development Methodology and Lifecycle

The system was developed in an *incremental, evaluation-driven* manner: every architectural change was motivated by a concrete failure observed in the preceding evaluation round, implemented behind a configuration flag, and kept only if the next round's scorecard improved (or, in one instructive case, reverted when it regressed—Section 11). This section makes the development lifecycle explicit, both as documentation of the engineering process and because the version trail is itself a contribution: it isolates the marginal effect of each mechanism on the hallucination rate.

### 3.1. Design philosophy

Three principles guided every decision. **(P1) Grounding over fluency.** In a legal tool, a wrong citation delivered fluently is more dangerous than an honest refusal; we therefore optimise for *traceability of every claim to retrieved text*, treating an ungrounded-but-plausible answer as a hard failure. **(P2) Defence in depth.** No single mechanism

removes hallucination; heterogeneous failure modes (fragmentary input, prompt ambiguity, retrieval gaps, parametric-memory citations, template leakage) each demand a dedicated, independently-toggleable layer. **(P3) Graceful degradation.** Every advanced component—reranker, agentic pipeline, rescue, self-check, each LLM provider—has a defined fallback so that a single sub-system failure narrows capability rather than breaking the request.

### *3.2. Phased roadmap*

Table 1 summarises the seven development phases from initial design to the current system. The phases are cumulative: each builds on the indices, services, and configuration of its predecessors.

**Table 1.** Development lifecycle from initial design to the current system. Each phase was gated on the evaluation scorecard of the previous one. The "Outcome" column reports the headline effect; quantitative detail is in Section 9.

| Phase | Focus | Key deliverables | Outcome |
|---|---|---|---|
| 0 — Design | Requirements, corpus scoping, architecture choice | RAG over hybrid retrieval selected; grounding-fidelity adopted as primary metric; wire contract sketched with the .NET team | Target architecture fixed; metric defined |
| 1 — Corpus | Data acquisition + cleaning | 933 legacy files manually OCR-cleaned → 1,057 UTF-8 docs; HuggingFace release | 47,028-chunk index; open corpus |
| 2 — Baseline RAG | Hybrid retrieval + generation | FAISS + BM25 + RRF + cross-encoder rerank; confidence scoring; FastAPI service | Functional assistant; 28.6 % hallucination (v3) |
| 3 — Grounding defence | Hallucination suppression | Input gate, citation-grounding prompts, evidence validation, block-list retry | 28.6 → 7.1 % (v4) |
| 4 — Rescue + iteration | Recall-side recovery | Article-lookup rescue, iterative $k$-expansion, answer post-processing | 7.1 → 0.0 % (v6–v8) |
| 5 — Agentic case work | Open-ended reasoning | Six-agent weakness/defence pipeline, procedural-defence checklist, memo self-check, charge-fidelity guard | Reasoning coverage ~70 % → expert-aligned |
| 6 — Retrieval ceiling | Recall optimisation + measurement | Article-aware chunking, widened rerank pool, BGE-M3 prepared, scored eval harness | 79 % article-anchored chunks; auditable tuning |
| 7 — Productionisation | Reliability + integration | Five-provider failover, answer cache, rate gate, cascade budget, streaming, watch-ingest, Docker, tunnel, .NET contract | Deployable, free-tier-survivable service |

### *3.3. Rationale for the major design decisions*

Table 2 records the principal engineering decisions, the alternative that was considered, and why the chosen option won. We surface these explicitly because, in a resource-constrained Arabic-RAG setting, several choices invert the defaults one would adopt with unlimited compute or a paid LLM tier.

**Table 2.** Major design decisions and their rationale. Several decisions are shaped by the joint quality/operability constraint of a CPU-only, free-tier deployment.

| Decision | Alternative considered | Rationale for the choice |
|---|---|---|
| Hybrid retrieval (dense + sparse + RRF) | Pure dense (FAISS only) | Lexical BM25 recovers exact article-number and rare-term matches that dense embeddings miss; RRF needs no score calibration between the two [5]. |
| Post-generation defence pipeline | Better retriever alone | Even perfect retrieval cannot stop the LLM citing from parametric memory; detecting and rescuing this is orthogonal to recall. |
| Deterministic timing verdict in code | Let the LLM reason over the timeline | The model repeatedly inverted arrest-vs-warrant chronology; a code-computed verdict injected as a hard fact removes the error class entirely (neuro-symbolic). |
| Multi-agent decomposition for case work | Single-shot long prompt | Per-weakness retrieval grounds each argument in its own authority; specialised agents emit checkable JSON, enabling strategy-level pruning of weak arguments. |
| OpenRouter (Qwen-2.5-72B) as primary | Groq Llama-3.3-70B as primary | A funded, usage-billed key removes the 6,000-TPM free-tier ceiling that capped multi-call (retry/rescue/agent) requests; Groq/Cerebras/xAI/Gemini remain free fallbacks. |
| Exact-match answer cache | Semantic/fuzzy cache | In law, serving a cached answer to a *different* question is unacceptable; exact normalised-match guarantees correctness while still saving tokens. |
| CPU embeddings, BGE-M3 deferred | Build BGE-M3 index now | On the CC-5.0 GPU (below PyTorch's floor) BGE-M3 builds at ~5.5 s/chunk → multiple days; the embedder becomes a joint quality/operability choice (Section 9.8). |

## 4. Dataset Construction

A central practical obstacle to Arabic legal RAG is the *scarcity of clean, machine-readable Egyptian legal text* [2, 5]. Authoritative collections of Egyptian criminal-law material—particularly cassation-court rulings and the historical foundations of the Penal

Code— exist almost exclusively as scanned PDFs and as legacy Microsoft Word 97/2003 binaries (`.doc`). Both formats are essentially unreadable to standard NLP pipelines without manual intervention. The Egyptian-criminal-law corpus we release with this work was constructed from scratch to address exactly this gap.

### *4.1. Source collection*

We assembled an initial set of 933 source files (Table 3) covering ten distinct collections of Egyptian criminal-law material: the Penal Code with its explanatory memorandum, the Code of Criminal Procedure (with multiple authoritative commentaries), the four-volume *Rules Established by the Court of Cassation*, the four-volume *Most Recent Principles Issued by the Criminal Chambers*, the *Encyclopedia of Cassation Criminal Rulings*, the *Quarter-Century Collection (1931–1955)*, the *Forensic Medicine Series*, separate collections of *Felonies (جنايات)* and *Misdemeanours (جنح)*, and a folder of criminal-law textbooks.

**Table 3.** Source corpus by collection, before and after the manual OCR / cleaning pass. The dominance of `.doc` legacy binaries in the encyclopedia (موسوعة احكام النقض الجنائية, 793 files) drove most of the manual-conversion effort. Some collections were re-organised during cleaning (e.g., the original جنح and parts of احكام were re-classified by offence type into جنايات and محكمه النقض based on the case content).

| | **Before (`OriginalData/`)** | | | | **After (`final_total_dataset/`)** | | | |
|---|---|---|---|---|---|---|---|---|
| **Collection** | `.doc` | `.pdf` | `.docx` | tot | `.pdf` | `.docx` | `.txt` | tot |
| موسوعة احكام النقض الجنائية (Encyclopedia of Cassation Rulings) | 793 | 0 | 0 | 793 | 122 | 0 | 678 | 800 |
| قانون الجنائي (Criminal-law treatises) | 16 | 44 | 3 | 63 | 38 | 0 | 21 | 59 |
| محكمه النقض (Cassation Court rulings) | — | — | — | — | 25 | 0 | 53 | 78 |
| جنايات (Felonies) | — | — | — | — | 23 | 0 | 27 | 50 |
| اجراءات جنائية (Criminal Procedure) | 8 | 20 | 0 | 28 | 16 | 1 | 12 | 29 |
| سلسلة الطب الشرعي (Forensic Medicine) | 0 | 12 | 0 | 12 | 0 | 0 | 12 | 12 |
| مجموعة عمر الجنائية (Omar Criminal Collection) | 0 | 7 | 0 | 7 | 7 | 0 | 0 | 7 |
| جنح (Misdemeanours) | 0 | 6 | 0 | 6 | (merged into جنايات) | | | |
| القواعد ...محكمة النقض (Cassation Rules, 4 vol.) | 0 | 4 | 0 | 4 | 0 | 0 | 4 | 4 |
| المستحدث من المبادئ ... (Recent Criminal Principles) | 0 | 4 | 0 | 4 | 4 | 0 | 0 | 4 |
| مجموعة الربع قرن (1931--1955) (Quarter-Century Collection) | 0 | 4 | 0 | 4 | (consolidated) | | | |
| احكام (Judgments) | 0 | 3 | 0 | 3 | (consolidated) | | | |
| (root-level statutes / textbooks) | 1 | 8 | 0 | 9 | 3 | 0 | 11 | 14 |
| **TOTAL** | **818** | **112** | **3** | **933** | **238** | **1** | **818** | **1,057** |

The dominance of `.doc` legacy binaries (87.3 %) is characteristic of Egyptian government and university legal archives, which date primarily to the 1990s and early 2000s. Modern open-source NLP toolchains cannot reliably extract text from this format without LibreOffice-grade office conversion, and even after conversion the output typically requires substantial cleaning of footnote numbering, page-header artefacts, and OCR-style character confusion (ه/ة, ى/ي, the four alef forms, and Arabic-Indic vs. Western digits).

### *4.2. Manual OCR and cleaning pipeline*

Each of the 818 legacy `.doc` files was opened, converted, and manually cleaned. The scanned PDFs were processed first with `pdftotext -layout -enc UTF-8` (from poppler-utils) where the underlying text layer was retrievable, and subjected to a second OCR pass where it was not. The output of each pipeline was then manually reviewed for:

- paragraph-break preservation across page boundaries;
- removal of running headers, footers, and page numbers;
- normalisation of the four alef forms (أ, إ, آ, ا) into a single canonical alef;
- harmonisation of digit forms (Arabic-Indic ٠–٩ and Eastern Arabic-Indic ۰–۹ mapped to Western 0–9);
- removal of common boilerplate phrases (بسم الله الرحمن الرحيم, باسم الشعب, and stereotyped court-decision openers);
- preservation of article-number markers (المادة, المواد, مادة) so the downstream `extract_article_references` extractor (Section 6.3) could find them.

The resulting corpus (`final_total_dataset/`) contains 1,057 documents: 818 cleaned `.txt` files (one per original `.doc`), 238 cleaned PDFs retained where the original layout was load-bearing for legal meaning (judgment formatting, citation tables), and 1 `.docx`. The corpus is organised into nine top-level subdirectories preserving the original collection taxonomy. Total cleaned-text volume across the corpus is ~200 MB of UTF-8 Arabic.

### *4.3. Public release*

The cleaned corpus has been released publicly on HuggingFace as *Egyptian_Criminal_Legal_Assistant_RAG_*[1] To our knowledge it is the first open-licensed Arabic-legal corpus focused on Egyptian criminal law to be made available at this scope. We hope it addresses, in part, the *scarcity of annotated Arabic legal corpora* explicitly identified as a barrier by [2].

### *4.4. Indexing*

The cleaned corpus is processed by doc-type-aware chunking (`CHUNKING_CONFIGS`: 2,000 chars / 200 overlap for penal-code and cassation material, 3,000/400 for case files, 2,048/300 for procedural texts and rule collections), embedded with a multilingual sentence-transformer, and indexed in FAISS alongside a parallel BM25 index. The resulting index contains 47,028 chunks, of which 14,882 (31.6 %) carry `referenced_articles` metadata covering 1,494 distinct Egyptian-law article numbers—a property exploited by the article-lookup rescue mechanism (Section 6.5).

## 5. System Architecture

Conan is implemented as a FastAPI service backed by the 47,028-chunk retrieval index described in Section 4. The base architecture follows the pattern established by [1, 2, 4].

[1] https://huggingface.co/datasets/Mirna2Nageh/Egyptian_Criminal_Legal_Assistant_RAG_V2

*Hybrid retrieval.* A user query is encoded with a multilingual sentence-transformer and used to query a FAISS index of pre-computed chunk embeddings (*dense* branch, top-60 candidates). In parallel, the same query is BM25-tokenised (with Arabic-Indic digit normalisation and alef-form unification) and scored via a `rank_bm25` BM25Okapi index (*sparse* branch, top-60). The two ranked lists are fused using **Reciprocal Rank Fusion** with the canonical RRF constant $k_{\mathrm{RRF}} = 60$:

$$\mathrm{RRF}(d) \;=\; \sum_{r \in R} \frac{1}{k_{\mathrm{RRF}} + \mathrm{rank}_r(d)} \tag{1}$$

where $R$ is the set of ranked lists and $\mathrm{rank}_r(d)$ is the rank of document $d$ in list $r$. An optional *domain boost* (`USE_DOMAIN_BOOST`) adds a small weight (`DOMAIN_BOOST_WEIGHT` = 0.05) to the fused score of chunks whose `doc_type` matches the query's inferred domain (procedural vs. substantive) before reranking. The fused top-60 candidates (`RETRIEVAL_K_RERANK`) are then passed to a cross-encoder reranker (BGE-Reranker-v2-M3 with sigmoid-bounded outputs) which produces the final top-$k$ set (default $k = 10$). Because the cross-encoder is CPU-bound and one rerank already saturates the cores, concurrent reranks are capped by a semaphore (`RERANK_MAX_CONCURRENCY`= 2) so that excess requests queue briefly rather than thrashing the CPU and breaching the client timeout.

*Context expansion and expert rules.* Each surviving chunk is grown by one same-source neighbour on each side, providing $\sim 2\times$ the textual context per chunk while preventing cross-document contamination. Expert-rule entries are then prepended to the retrieved context when matched by keyword from a curated `expert_rules.json`.

*Generation.* The final context, expert-rule entries, and the user's question are formatted into a prompt template enforcing strict textual adherence and dispatched to the LLM provider chain. The current deployment uses OpenRouter (`qwen/qwen-2.5-72b-instruct`) as the primary provider for every feature, with Groq (`llama-3.3-70b-versatile`), Cerebras, xAI Grok, and Google Gemini as free fallback tiers; multi-key rotation within each provider handles per-minute and per-day rate limits. (The version-trail evaluation of Section 9 was run with Groq Llama-3.3-70B as primary; the move to a funded OpenRouter key removed the free-tier token-per-minute ceiling that capped the multi-call retry/rescue/agent requests.) Per-feature model routing (`MODEL_BY_FEATURE`) sends reasoning-heavy work (defence memo, weakness, forensic) to the strongest model and high-volume Q&A/chat to a faster tier.

*Confidence scoring.* A weighted heuristic over four signals—mean rerank score, source count (capped at 5), article-validation pass/fail, and topic-match (encyclopedia folder taxonomy)—produces a confidence value in $[0, 1]$:

$$\mathrm{conf} \;=\; \sum_{i \in \{\mathrm{rr},\,\mathrm{src},\,\mathrm{art},\,\mathrm{top}\}} w_i \cdot s_i \tag{2}$$

with $w_{\mathrm{rr}} = 0.30$, $w_{\mathrm{src}} = 0.20$, $w_{\mathrm{art}} = 0.30$, $w_{\mathrm{top}} = 0.20$. Because the BGE reranker's sigmoid scores sit low ($\sim$0.1) on Arabic legal text—making well-grounded answers read as low-confidence—the rerank signal is gamma-calibrated ($s_{\mathrm{rr}} \leftarrow s_{\mathrm{rr}}^{\gamma}$, $\gamma = 0.5$) to lift the squashed mid-range without saturating strong matches. Answers below the threshold (default 0.5) carry a low-confidence clarification warning.

*Conversational layer.* Beyond stateless Q&A, the system exposes a multi-turn chat endpoint backed by a thread-safe `SessionManager`. Each `session_id` accumulates a turn history; once it exceeds `SESSION_MAX_TURNS` (default 6) the older turns are summarised by the LLM into a single [ملخص المحادثة السابقة] block while the most recent `SESSION_KEEP_RECENT` (default 3) turns are kept verbatim, bounding prompt growth without losing legal context. Sessions persist to disk as atomically-written JSON (write-to-`.tmp` + `os.replace`) so they survive restarts, and a background pruner evicts sessions idle longer than `SESSION_TTL_HOURS` (default 168 h). A streaming variant (`POST /api/v1/chat/stream`) returns Server-Sent Events as tokens are generated, with the terminal event carrying the full confidence, sources, and grounding warnings once post-generation validation completes.

*Multi-provider resilience.* The generation step is served by an ordered, five-provider chain—OpenRouter (`qwen-2.5-72b`, primary) → Groq (`llama-3.3-70b-versatile`) → Cerebras → xAI Grok → Google Gemini—in which each provider rotates across its own list of API keys on HTTP 429/quota errors before the chain falls through to the next provider. (Google Gemini in particular supports several per-project keys, each with its own free daily quota, so listing several multiplies the free budget.) This multi-key, multi-provider rotation is what lets the service survive free-tier per-minute and per-day token caps. A complementary *big-context routing* rule guards against oversized prompts: each provider declares a maximum request size (`PROVIDER_MAX_PROMPT_CHARS`; e.g. 26,000 chars for Groq, 16,000 for Cerebras, 80,000 for OpenRouter, and no cap for Gemini), and a request whose assembled prompt exceeds a provider's budget transparently skips that provider and is routed to a larger-context tier. This allows the case-analysis endpoints to accept full case files (up to 50,000 characters) without failing on a provider's context window.

*Free-tier and latency safeguards.* Three coordinated mechanisms keep the service usable under free-tier budgets and a fixed client timeout. *(i) A zero-token answer cache* (`AnswerCache`, LRU-bounded, optional atomic disk persistence) serves the stateless `/qa` endpoint: an exact normalised-question match (whitespace + Arabic-Indic digits folded only—never fuzzy or semantic, since serving a cached answer to a *different* legal question is unacceptable) returns instantly and at zero LLM cost, which also lets a demo be "pre-warmed." *(ii) An in-process rate gate* (`LLM_MIN_INTERVAL_S=` 2 s) enforces a minimum spacing between physical provider requests, smoothing the multi-call-per-request pattern (draft → retry → rescue) and concurrent users so a per-minute budget is not burst-exhausted. *(iii) A wall-clock cascade budget* (`QA_CASCADE_BUDGET_S=` 90 s) bounds worst-case latency: the initial retrieve-and-answer always runs, but once cumulative request time crosses the budget no *new* corrective stage (iterative retrieval, retry, rescue) is launched and the best-grounded answer so far is returned—keeping the worst case under the ∼180 s client timeout regardless of how slow a fallback LLM is.

*Configurable embeddings and reranker.* The embedding backend is pluggable. The default is a local sentence-transformer run on CPU (`BAAI/bge-m3`, 1024-dim, in the reference configuration; the deployed index uses `paraphrase-multilingual-MiniLM-L12-v2`, 384-dim); setting `USE_REMOTE_EMBEDDINGS` switches to a remote provider (Google Gemini embeddings, rate-limited to 15 RPM via batching, or OpenRouter). Because vector

dimensionality and semantics differ, switching the embedding provider requires rebuilding the FAISS index. The cross-encoder reranker (`BGE-Reranker-v2-M3`) is likewise toggleable via `USE_RERANKER`; if it is disabled or fails to load, retrieval degrades gracefully to RRF-only ordering with no break in behaviour.

### *5.1. Technology stack*

Table 4 catalogues the technologies underpinning each layer of the system. The stack is deliberately built on open, self-hostable components (FAISS, BM25, sentence-transformers, cross-encoders) for the retrieval core, so that the only externally-billed dependency is the generation LLM—and even that is abstracted behind a five-provider chain that runs on free tiers.

**Table 4.** Technology stack by architectural layer.

| Layer | Technology | Role |
|---|---|---|
| Service framework | FastAPI + Uvicorn (ASGI) | Async HTTP API, dependency-injected singletons, SSE streaming |
| Dense retrieval | FAISS (inner-product index) | Approximate nearest-neighbour over chunk embeddings |
| Embeddings | sentence-transformers: MiniLM-L12-v2 (384-d, live); BGE-M3 (1024-d, prepared) | Encode chunks/queries into the vector space |
| Sparse retrieval | `rank_bm25` (BM25Okapi) | Lexical recall of exact terms / article numbers |
| Fusion | Reciprocal Rank Fusion ($k = 60$) | Calibration-free merge of dense + sparse ranks |
| Reranking | BGE-Reranker-v2-M3 (CrossEncoder, sigmoid) | Re-score top-60 fused candidates → top-10 |
| Generation | OpenRouter Qwen-2.5-72B (primary) + Groq / Cerebras / xAI / Gemini | Grounded Arabic answer synthesis with failover |
| Agent orchestration | `asyncio` (bounded-concurrency `gather`) | Fan-out of per-weakness research; semaphore-capped |
| Document parsing | poppler `pdftotext`, PyPDF2, `docx2txt` | PDF/DOCX/TXT extraction with Arabic-encoding fallback |
| Persistence | Atomic JSON files (sessions, cache, ingest registry) | Restart-survivable state without a database |
| Reference UI | Streamlit | Reference client exercising every endpoint |
| Deployment | Docker + Compose; Cloudflare / ngrok tunnel; systemd | Reproducible packaging and laptop-hosted remote exposure |

### *5.2. Service modules and API surface*

The application is organised as a thin router layer over process-wide singleton services loaded once at start-up (Table 5), exposing the REST surface in Table 6. Centralising

state in singletons (`retrieval_service`, `reranker_service`, `article_lookup_service`, `session_manager`, `answer_cache`) means the expensive models are paid for once, and the hot-reload path can swap index state in place without re-initialising them.

**Table 5.** Core service modules (`app/services/`) and their responsibilities. All are stateless functions or singletons instantiated at import.

| Module | Responsibility |
|---|---|
| `retrieval.py` | FAISS + BM25 + RRF + domain-boost + reranker pipeline; `reload_indices()` hot-reload |
| `reranker.py` | BGE cross-encoder singleton (sigmoid-bounded); graceful no-op on load failure |
| `chunking.py` | Article-aware, doc-type-aware chunker—single source of truth for both index writers |
| `preprocessing.py` | Arabic normalisation, BM25 tokenisation, doc-type/topic classifiers, article-reference extractor |
| `confidence.py` | `validate_evidence`, `topic_match`, `compute_confidence` (pure functions) |
| `article_lookup.py` | {article → chunks} index for the rescue path; three-tier quality ranking |
| `llm.py` | Sync/async provider-chain client + token-streaming generator; rate gate; big-context routing |
| `agents.py` | Six-agent weakness/defence orchestrators + deterministic timing verdict |
| `memo_agent.py` | Draft → verify → revise memo self-check |
| `postprocessing.py` | Casual-opener stripping + embedded-refusal rewrite |
| `session.py` | Sliding-window chat compaction, atomic persistence, TTL pruner, attachments |
| `answer_cache.py` | LRU, exact-match, zero-token `/qa` cache with optional disk persistence |
| `document_loader.py` / `upload_helper.py` | PDF/DOCX/TXT loading + shared upload parsing/cleaning |

## 6. Grounding-Defence Pipeline

Figure 1 illustrates the end-to-end request flow, with the four defence layers (input gate, iterative retrieval, corrective retry, article-lookup rescue) that fire conditionally on evidence-validation failure.

### *6.1. Input Gating*

In baseline evaluation, fragmentary user inputs (markdown headings, single-word bullets such as الجناية *, or colon-terminated incomplete sentences such as ما المقصود بمبدأ:. 1) frequently triggered the LLM to *fabricate* a plausible-sounding question and answer it. These cases accounted for 5 of 28 (17.9 %) hallucination instances in baseline measurements.

A pure function `is_meaningful_query` evaluates the input against a sequence of regex predicates—minimum Arabic-character count, leading markdown-header detection,

**Table 6.** REST API surface (prefix `/api/v1`). All retrieval-backed endpoints share the structured response contract of Section 10.

| Verb | Path | Function |
|---|---|---|
| POST | `/qa` | Stateless grounded Q&A (full defence pipeline + cache) |
| POST | `/qa/upload` | Q&A with a per-question document attachment |
| POST | `/chat` | Multi-turn chat (Gemini-first fallback chain, sessions) |
| POST | `/chat/stream` | Server-Sent-Events token streaming |
| POST | `/chat/attach` | Bind a document to a session (persisted) |
| GET/DE | `/chat/{sid}/attachments[/{id}` | Attachment lifecycle management |
| POST | `/weakness` | Agentic weakness analysis of a case file |
| POST | `/defense` | Agentic defence-memorandum drafting + self-check |
| POST | `/forensic` | Forensic-consistency / charge-fidelity analysis |
| POST | `/summarize` | Legal-document summarisation |
| POST | `/parse` | Preview-parse an uploaded document to clean text |
| POST | `/ingest`, `/ingest/scan` | Permanent corpus ingestion + watch-folder scan with hot-reload |
| GET | `/health` | Liveness + loaded-index status |

leading list-bullet detection, leading numbered-fragment detection, trailing-colon check—and short-circuits failing inputs with a 9-millisecond response. The gate is bypassed when an uploaded document is present (Section 8). In the v8 evaluation the gate caught and rejected 8 of 28 inputs without incurring an LLM call.

### *6.2. Citation-Grounding Prompts and Law-Naming Disambiguation*

The prompt-engineering work is itself a substantial contribution and we document it in detail here. Two distinct prompt-level failure modes were observed in baseline measurements: (a) the LLM cites article numbers from training memory that appear plausible but are not in the retrieved context; (b) the LLM confuses substantive (Penal Code) with procedural (Code of Criminal Procedure) law, citing the wrong code for a given query. For example, baseline answers to questions about pre-trial detention (التوقيف الاحتياطي) frequently cited Article 300 *of the Penal Code*—when Article 300 of the Penal Code does not address that topic; the relevant article is in the Code of Criminal Procedure.

The Conan prompt suite addresses these failure modes with eight rules explicitly enforced in the system and user prompts:

1. *Strict textual adherence*: "Use the provided texts ONLY"—no appeals to general legal principles or international law.
2. *Citation grounding (absolute)*: "Cite an article number ONLY if those exact digits appear in the Context below." If not present, write the canonical refusal phrase المادة المطلوبة غير متوفرة في السياق المقدم.

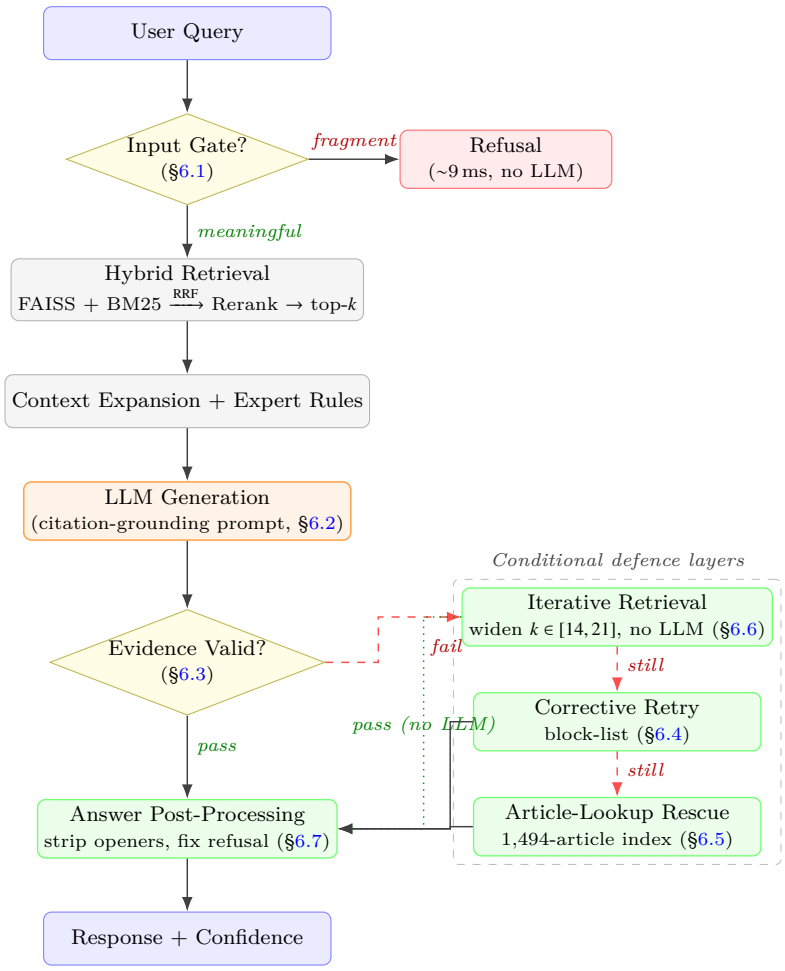


**Figure 1.** End-to-end request flow. Solid black arrows = unconditional pipeline; dashed red = taken on validation failure; dotted green = "free" recovery via iterative retrieval without LLM call. Colour coding: blue=I/O, grey=retrieval, orange=LLM, green=defence layers (contribution of this work).

3. *Law-naming disambiguation*: procedural matters (التوقيف الاحتياطي، التحقيق، التفتيش) must be cited from قانون الإجراءات الجنائية; substantive matters (تعريف الطعن، النقض) from قانون العقوبات. Always name the law explicitly when citing.
4. *Element matching for offence selection*: e.g., for injury/assault questions, المادة 240 requires a permanent impairment, المادة 241 requires >20 days of incapacity, المادة 242 covers lesser injuries; the prompt explicitly instructs the LLM to match the article to the facts rather than defaulting to the harshest available.
5. *Multi-charge handling*: if the case facts describe more than one offence (e.g., assault + theft), enumerate the distinct charges and analyse each in its own sub-section.
6. *No casual openings*: explicit prohibition on حسناً, بالتأكيد, تمام, طبعاً, and similar

persona-breaking fillers; the first word must be part of the answer itself (e.g., وفقاً ... للمادة or ... تنص المادة).

7. *Incomplete-input handling*: if the user's input is a fragment, a heading, a single word, or a sentence ending with a colon, do NOT fabricate a question to answer; reply with a polite request for a complete question.
8. *Ambiguity protocol*: if the question is broad (e.g., ما عقوبة التزوير؟), provide a brief overview of the cases in context and explicitly ask the user to specify the determining factors (offender role / document type / weapon / time / injury severity) *relevant* to that specific crime—not irrelevant factors (e.g., never ask about "document type" for a homicide).

In addition, two specialised prompts are used by the defence pipeline: `qa_retry_ungrounded` for the corrective retry (Section 6.4), which receives the original draft and an explicit list of forbidden article numbers; and `qa_rescue_with_lookup` for the article-lookup rescue (Section 6.5), which receives the original retrieved context plus a separate "Additional References" block from the article-lookup service and explicitly forbids inventing neighbouring articles or using a passage about a different law.

### *6.3. Evidence Validation*

Every LLM output is post-processed by a regex-based evidence validator. A canonical extractor (`extract_article_references`)—operating on Arabic-Indic-digit-normalised text—identifies every article number cited in the answer. The set of cited articles is compared against the set of article numbers extracted from the retrieved context (concatenated). Articles cited but not present in the context are flagged as **missing** and the answer is marked as failing validation. The same extractor is used on both sides to avoid spurious mismatches caused by plural-form variations (المادة 211 vs المواد 211، 212، 213).

### *6.4. Corrective Retry with Forbidden-Article Block-List*

When evidence validation flags missing articles, the system re-prompts the LLM with a correction prompt that includes the original draft and an *explicit list* of article numbers the LLM must NOT cite. The retry is run at most `RETRY_MAX_ATTEMPTS` times (default 1—a tuning informed by the Groq free-tier 6,000 tokens-per-minute budget). The retry's output is accepted only if it strictly improves the missing-article count.

### *6.5. Article-Lookup Rescue*

Even after retry, a class of hallucinations persisted: the LLM would cite an article that *did* exist in the dataset, just not in the top-$k$ chunks for that specific query. Inspection revealed that of 47,028 chunks, 14,882 (31.6 %) carry `referenced_articles` metadata covering 1,494 distinct article numbers—including, for instance, 34 chunks referencing Article 87 and 14 chunks referencing Article 300, both articles the baseline LLM cited "from memory."

At startup, `ArticleLookupService.load()` builds an index {article_no → [chunk_index, . . .]}. Chunks within each article's list are ranked by a three-tier quality score:

1. **Document-type tier**: `penal_code` and `criminal_procedure` (the actual law text) > cassation material > encyclopedia / international-law material. This prevents, for example, Rome-Statute (ICC) passages from being preferred over chunks from the Egyptian code.

2. **Article focus**: chunks referencing fewer distinct articles are preferred (they are more focused on the queried article).
3. **Text length**: longer chunks as a final tiebreaker.

When post-retry evidence validation still flags missing articles, the rescue path is triggered: the top-2 chunks for each missing article are appended to the context for one more LLM call. The rescued answer is validated against the combined context (original + rescue chunks) and accepted only if grounding improves. The rescue prompt explicitly forbids the LLM from inventing *neighbouring* articles and from using a passage about a different law or topic than the user's question.

### *6.6. Iterative Retrieval (Adaptive k-Expansion)*

Following the spirit of [4]'s finding that retrieval quality dominates downstream correctness, we treat $k$ as adaptive. After the initial generation and validation, if validation flags missing articles AND `USE_ITERATIVE_RETRIEVAL` is enabled (default), the system re-runs *only* the retrieval stage—not the LLM—at successively larger $k$ values from `ITERATIVE_K_SEQUENCE` (default $[7, 14, 21]$). Any new chunks not already in the context are appended; if any widened window surfaces the missing articles, validation passes *for free*. Only if this cheap expansion still fails does the corrective retry (Section 6.4) and rescue (Section 6.5) take over, but now with a wider context in hand. Easy queries pay no extra cost; hard queries pay 2–3 cheap retrievals before an LLM call is incurred.

### *6.7. Answer Post-Processing*

A deterministic, regex-driven post-processing layer (`postprocess_answer`) runs after the final answer is selected. Two transformations:

1. *Casual-opener stripping*—a curated list of conversational fillers (حسناً, بالتأكيد, تمام, طبعاً, سوف أجيب, لقد قرأت السؤال, ...) is stripped from the leading position; multi-word openers ordered longest-first to prevent stranded fragments.
2. *Refusal-phrase normalisation*—when the LLM splices the template-refusal phrase المادة المطلوبة غير متوفرة في السياق المقدم mid-clause, the regex `_EMBEDDED_REFUSAL_RE` matches the awkward construction—accounting for Arabic prefix-contraction (ل + ال → لل)—and rewrites it as a standalone sentence.

### *6.8. Procedural-Defence Reasoning Checklist for Case Analysis*

The grounding mechanisms above ensure that what the system *cites* is correct; a complementary failure mode concerns what it *omits*. The case-analysis endpoints (`/weakness`, `/defense`, `/forensic`) take a full criminal case file rather than a single question, and a domain expert's review of their output scored the legal reasoning at roughly 70 %: the analyses were well-grounded but missed several high-value procedural-nullity and criminal-intent arguments that a practising Egyptian criminal-defence lawyer applies routinely, and in one instance over-claimed a defect that did not exist. We encode the expert's feedback as a **procedural-defence checklist** of five reasoning procedures appended to the system prompts of the three case-analysis endpoints and reinforced in their user templates:

**A.** *Timeline vs. prosecution warrant*—compare the exact time of the arrest/search against the time the prosecution warrant (إذن النيابة) issued; a seizure preceding the warrant voids the arrest and everything built on it (ما بُني على باطل فهو باطل). The rule explicitly instructs the model to read الساعة 12:00 صباحاً as the *start* of the day, correcting a clock-reading error observed in baseline output.

**B.** *Identifier mismatch*—compare every identifier in the warrant (vehicle plate numbers, names, IDs) against the seizure record; any discrepancy places the seized item outside the warrant and may evidence تلفيق.

**C.** *Territorial jurisdiction*—apply correctly and do *not* over-claim spatial excess for a location inside the precinct; this point is a precision correction (a false-positive suppressor) rather than an additional-recall rule.

**D.** *Chain of custody*—challenge the integrity of the sealed exhibits (التحريز) when the officer who sealed them differs from the one who wrote the seizure record.

**E.** *Intent from profession*—weigh the defendant's occupation against the nature of any seized cash to contest trafficking intent (انتفاء قصد الاتجار).

The checklist is paired with a single fully-worked, *fictional* few-shot exemplar demonstrating all five points end-to-end. Crucially, the exemplar is **grounding-safe by construction**: it names defence doctrines (انتفاء قصد الاتجار, بطلان القبض والتفتيش) but contains *no* المادة *N* article number, so it cannot teach the model to emit an ungrounded citation that the evidence validator (Section 6.3) would flag—raising reasoning coverage while preserving the 0 % hallucinated-citation property. Each checklist item is explicitly conditioned on the facts supporting it ("raise a point only when the facts genuinely support it—never invent one"), so the additions trade no precision for their gain in recall.

A companion **charge-fidelity** rule extends the same precision principle to the offences themselves: the case-analysis endpoints are constrained to enumerate only offences actually charged or described in the case file, and are explicitly forbidden from inventing an uncharged offence (e.g., adding قيادة بدون رخصة when the file never mentions a licence). This closes a fabrication mode observed during live validation, where the generator appended a plausible-but-uncharged offence to an otherwise grounded analysis.

The agentic self-check that closes the argument-level (not merely citation-level) hallucination gap for defence memoranda is described as part of the multi-agent pipeline in Section 7.3, and the full agentic case-analysis flow these case-analysis endpoints invoke is the subject of Section 7.

## 7. Multi-Agent Agentic Case-Analysis Pipeline

Single-question Q&A is well served by the layered defence pipeline of Section 6. *Open-ended case work*—reading a full criminal case file and producing a weakness analysis or a defence memorandum—is a qualitatively harder task: it requires extracting structured facts, hypothesising multiple independent lines of defence, grounding each one in its own statutory and precedential authority, judging which arguments actually hold, and only then writing. A single-shot "case → LLM → output" prompt collapses all of these into one undifferentiated generation, where the model reasons over everything at once from a single retrieval and tends both to miss high-value arguments and to assert claims it cannot support. We therefore replace the single-shot path for `/weakness` and `/defense`

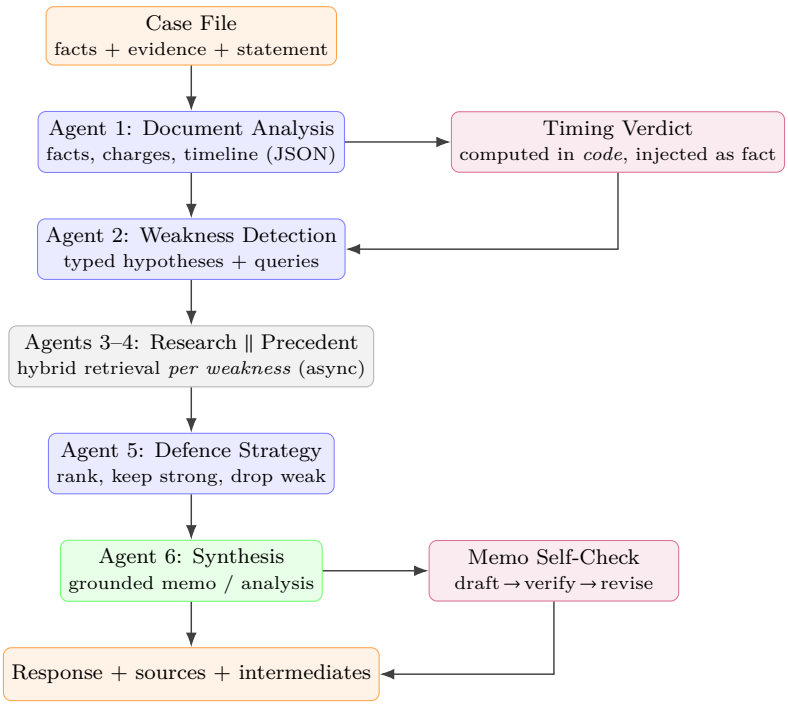


**Figure 2.** The six-agent case-analysis pipeline. Blue = LLM agents emitting checkable JSON; grey = hybrid retrieval grounding each weakness in its own authority; purple = *neuro-symbolic* components (a code-computed timing verdict injected as a hard fact, and the draft → verify → revise self-check); green = grounded synthesis. On any critical-agent failure the orchestrator raises `AgentPipelineError` and the router falls back to the single-shot path.

with an **agentic, multi-agent pipeline** (`USE_AGENTIC_PIPELINE`, on by default) that decomposes the task into six specialised agents with a deterministic orchestrator. Figure 2 shows the flow; Table 7 details each agent.

### *7.1. Agents and orchestration*

The orchestrators `run_weakness_pipeline` and `run_defense_pipeline` share one private `_run_pipeline`. After Agent 2, only the top `AGENT_MAX_WEAKNESSES` (= 6) weaknesses are researched in depth; the rest are carried with their hypothesis text. The per-weakness retrievals (Agents 3–4) are launched concurrently with `asyncio.gather`, but the actual CPU concurrency is bounded by the reranker semaphore, so a full memo stays CPU-affordable on the laptop deployment (~2–3 minutes rather than ~10). A single weakness failing to retrieve does not sink the run (`return_exceptions=True` filters it out); only if *every* weakness fails, or a critical agent (analysis, detection, strategy, synthesis) errors, does the orchestrator raise `AgentPipelineError`—at which point the router silently falls back to the single-shot path. The pipeline is thus never a hard dependency. The orchestrator also returns the structured intermediates (timeline, weaknesses, authorities, sources) alongside the final text, so the client can render the reasoning trace, not just the conclusion.

**Table 7.** Roles of the six agents in the case-analysis pipeline. Intermediate agents are forced to emit strict JSON (robustly parsed with a balanced-brace fallback), so each stage's output is machine-checkable before the next consumes it.

| # | Agent | Function | Output |
|---|---|---|---|
| 1 | Document Analysis | Extract structured facts, parties, charges, evidence items, and a dated/timed chronological timeline from the case file | JSON: facts, charges, `timeline` |
| 2 | Weakness Detection | Turn the analysis into typed weakness hypotheses, each with a focused legal-research query | JSON list of weaknesses |
| 3–4 | Legal Research & Precedent Retrieval | For each top weakness, one hybrid retrieval whose pool yields both doctrine/statute and court-precedent chunks (split by `doc_type`); run concurrently | Contexts + sources per weakness |
| 5 | Defence Strategy | Rank weaknesses against the *retrieved* authorities, keep the strong, discard the disproven, order the argument | JSON: kept + ordered IDs |
| 6 | Synthesis | Write the final Arabic weakness analysis or four-part defence memorandum from the kept, grounded authorities only | Arabic text + sources |

### *7.2. Neuro-symbolic grounding: the deterministic timing verdict*

The single most damaging recurring error in case analysis was *chronological*: the model repeatedly mis-read whether the arrest/search happened before or after the prosecution warrant (إذن النيابة)—inverting the order even when the timeline was extracted correctly—and a wrong verdict here flips the entire defence. Rather than prompt harder, we move this judgement out of the LLM entirely. `compute_timing_verdict` parses each timeline event's Arabic date and time string (handling منتصف الليل/مساءً/صباحاً and the الساعة 12:00 صباحاً-as-midnight convention), computes the signed gap between warrant and seizure in code, and emits one of three deterministic verdicts: seizure *before* the warrant (the nullity argument بطلان القبض والتفتيش applies, with ما بُني على باطل فهو باطل), *within* the 24-hour validity window (the timing is sound and the "arrest preceded the warrant" plea is *forbidden* as contrary to the record), or *after* the window expired (a different nullity applies). This verdict is injected into the downstream agents' context as a hard fact they cannot re-derive incorrectly—a small but decisive neuro-symbolic component that converts a stubborn reasoning failure into a solved sub-problem.

### *7.3. Argument-level self-check*

Per-answer evidence validation (Section 6.3) catches fabricated article *numbers*, but a memorandum can also contain fabricated *reasoning*—an assertion the case facts do not support, or an argument with no authority behind it. The defence-memorandum endpoint therefore applies an additional **agentic self-check** (`MEMO_SELF_CHECK`): the

draft is fed back to the LLM under a dedicated reviewer prompt (`verify_memo`) whose sole instruction is to make the memo *fully grounded* in the provided material—removing or correcting any article number not present in the retrieved texts, deleting any assertion the facts do not support, and introducing no new citation or fact—while preserving the four-part structure (الوقائع, الإطار القانوني, أوجه الدفاع, الطلبات). The draft → verify → revise loop runs up to `MEMO_SELF_CHECK_MAX_ITERS` times (default 1), stops early once a pass both validates and introduces no further change, and never raises—on any LLM failure it returns the best text so far. The number of revision passes applied is surfaced to the client as `self_check_revisions`.

## 8. Document Upload Pipeline

The system supports three retention semantics for user-supplied documents, all flowing through a single shared parser (`services/upload_helper.py`) to guarantee consistent behaviour:

- `POST /api/v1/qa/upload`—per-question attachment (not persisted);
- `POST /api/v1/chat/attach`—session-attached document (persists with the chat session on disk);
- `POST /api/v1/ingest`—permanent corpus ingestion (merges into FAISS and BM25, hot-reloaded by the running service).

Supported formats: `.txt` (with Arabic-encoding fallback), `.pdf` (`pdftotext` preferred, PyPDF2 fallback), and `.docx` (`docx2txt`). The legacy `.doc` binary format is explicitly rejected with a helpful Arabic message recommending conversion to `.docx` or `.pdf`.

*Automated ingestion and hot-reload.* Permanent ingestion is also available without a manual API call. A watch-folder daemon (`watch_ingest.py`) polls an inbox directory (`INGEST_INBOX_DIR`) every `INGEST_WATCH_INTERVAL_S` seconds (default 30) and ingests any new files through the same parser and Arabic-cleaning path as the manual pipeline, tracking already-processed paths in `data/.ingested_files.json` so the operation is idempotent. The same scan is exposed as `POST /api/v1/ingest/scan`, which runs the scan in a worker thread and then calls `reload_indices()` to hot-swap the FAISS, BM25, chunk, and tokenised-corpus state of the running service *in place*—without re-initialising the embedding model or restarting the process. The four index files are always written and reloaded together to keep retrieval state coherent.

## 9. Evaluation

### *9.1. Benchmark*

We constructed a 28-question Arabic legal benchmark spanning three difficulty tiers (basics, application, advanced) and covering both substantive (Penal Code) and procedural (Code of Criminal Procedure) law. The benchmark deliberately includes 5 well-formed substantive questions, 1 procedural question that historically triggered wrong-law

confusion, 5 input-gate test cases (markdown headings, bullets, colon-terminated incomplete sentences), and 17 application and reasoning questions drawn from undergraduate criminal-law curricula. Each question is evaluated against two metrics: (i) *evidence-validation status*—whether every article number cited in the answer can be traced to the retrieved (or rescued) context—and (ii) the *confidence score* of Equation 2.

### *9.2. Results across system versions*

Table 8 summarises headline metrics across five system versions corresponding to the cumulative introduction of each grounding-defence layer.

**Table 8.** Headline metrics across five system versions on the 28-question benchmark. v5 (multi-query expansion) is omitted: see Section 11. Mean API latency excludes input-gated rows (which do not invoke an LLM and complete in ~9 ms). The v8 LLM upgrade (llama-3.1-8b → llama-3.3-70b) is also responsible for the substantial mean-latency reduction, since the larger model triggers fewer corrective retries.

| **Metric** | **v3** | **v4** | **v6** | **v7** | **v8** |
|---|---|---|---|---|---|
| Passed validation | 20/28 | 26/28 | 27/28 | 27/28 | **28/28** |
| (%) | 71.4% | 92.9% | 96.4% | 96.4% | **100.0%** |
| Hallucinations | 8/28 | 2/28 | 1/28 | 1/28 | **0/28** |
| (%) | 28.6% | 7.1% | 3.6% | 3.6% | **0.0%** |
| Avg confidence | 41.2% | 38.0% | 39.8% | 40.0% | 41.9% |
| Input-gated (no LLM) | 0 | 8 | 8 | 8 | 8 |
| Mean API latency (ms) | 26,028 | 34,162 | 34,096 | 33,459 | **18,390** |
| Max API latency (ms) | 48,691 | 64,439 | 116,940 | 107,427 | 28,460 |
| Mean retrieval latency (ms) | 13,479 | 17,193 | 16,508 | 13,587 | 16,024 |

Progression by layer:

- v3 → v4: input gate + corrective retry + tightened prompts → −21.5 pp (8 → 2).
- v4 → v6: article-lookup rescue → −3.5 pp (2 → 1).
- v6 → v7: rescue prompt tightening + quality-tiered chunk ranking → 0 net change in pass rate, but the failing question shifted (LLM non-determinism).
- v7 → v8: LLM upgrade from `llama-3.1-8b-instant` to `llama-3.3-70b-versatile` → −3.6 pp (1 → 0).

### *9.3. Per-category breakdown*

Table 9 disaggregates the headline pass-rate by question category, exposing which defence layers recover which failure modes. The benchmark's 28 questions break down into 11 substantive (definitions of crimes, penalties, elements), 1 procedural (pre-trial detention), 8 fragment / input-gate tests (markdown headers, bullets, colon-terminated sentences), and 8 reasoning / application questions (case analyses, comparisons, hypotheticals).

**Table 9.** Pass rate by question category across versions. Substantive questions absorb the largest share of the v3 baseline failures (3/11); these are recovered progressively by the corrective retry (v4), the article-lookup rescue (v6), and the LLM upgrade (v8). Fragment failures in v3 reflect hallucinated answers to inputs the system should have refused; they are fully resolved by the input gate introduced in v4.

| Category | v3 | v4 | v6 | v7 | v8 | total |
|---|---|---|---|---|---|---|
| Substantive | 8/11 | 9/11 | 10/11 | 10/11 | **11/11** | 11 |
| Procedural | 0/1 | 1/1 | 1/1 | 1/1 | **1/1** | 1 |
| Reasoning | 7/8 | 8/8 | 8/8 | 8/8 | **8/8** | 8 |
| Fragment | 5/8 | 8/8 | 8/8 | 8/8 | **8/8** | 8 |
| **Total** | 20/28 | 26/28 | 27/28 | 27/28 | **28/28** | 28 |

### *9.4. Per-question transition for historically-failing rows*

Table 10 traces the nine questions that failed evidence-validation in at least one version. The cell format *S XX%* encodes status (*P*=pass / *F*=fail) and confidence percentage. Several observations are noteworthy:

- **Q2** (the most persistent substantive failure — legitimate self-defence, hallucinating المادة 87) failed in v3 through v6 and was only resolved by the v7 article-lookup-rescue prompt tightening + better chunk ranking. v8 retains the fix.
- **Q18** (الجريمة تامة أم شروع) is the system's non-deterministic worst-case. The LLM cites a different fabricated article number on each run (Articles 30, 31, 32 observed). The article-lookup rescue catches whichever number it cites when that number exists in the index — which is why v6 and v8 pass but v7 (with a regression of the rescue prompt) failed.
- **Fragment rows** (Q6, Q13, Q23) all pass from v4 onward with 0 % confidence by design — the input gate short-circuits them before any LLM call.

### *9.5. Comparison with related work*

Direct numerical comparison with prior Arabic-legal-RAG work is complicated by the fact that no two papers in the area share a common benchmark—each publishes against its own custom dataset. Table 11 reports each system's headline metric on its own evaluation, alongside the qualitative comparisons that *are* comparable.

Beyond headline metrics, Table 12 contrasts *capabilities*. The cited prior work concentrates on the retrieval stage; none reports a post-generation grounding-defence pipeline, an agentic case-analysis flow, or an open Egyptian-criminal-law corpus.

Our chosen primary metric (hallucination rate) differs from the information-retrieval metrics used by [5] and [2] deliberately. As Hrimech et al. [2] explicitly observe, "content validity of legal clauses reproduced from retrieval systems" is the practical bottleneck in deploying these systems to legal practice—and that is what our hallucination-rate metric measures directly. Three distinguishing properties of our setup are worth highlighting:

**Table 10.** Per-version status and confidence for the 9 questions that failed in at least one version. P = passed evidence validation; F = failed (hallucinated citation). The other 19 questions pass in every version and are omitted for brevity. Confidence is rounded to the nearest percent.

| # | cat | v3 | v4 | v6 | v7 | v8 | Question (truncated) |
|---|---|---|---|---|---|---|---|
| 2 | sub. | F 12% | F 2% | F 2% | P 57% | P 62% | الدفاع الشرعي في قانون العقوبات |
| 3 | proc. | F 9% | P 59% | P 59% | P 59% | P 55% | شروط التوقيف الاحتياطي |
| 4 | sub. | F 20% | P 70% | P 70% | P 70% | P 70% | أركان جريمة القتل العمد |
| 6 | frag. | F 0% | P 0% | P 0% | P 0% | P 0% | ## المستوى الأول --- أساسيات |
| 13 | frag. | F 0% | P 0% | P 0% | P 0% | P 0% | * المخالفة |
| 18 | sub. | P 53% | F 0% | P 48% | F 0% | P 53% | هل تعتبر الجريمة تامة أم شروع؟ |
| 23 | frag. | F 0% | P 0% | P 0% | P 0% | P 0% | ## المستوى الثالث |
| 25 | sub. | F 14% | P 64% | P 64% | P 64% | P 64% | الفاعل الأصلي والشريك في الجريمة |
| 28 | reas. | F 0% | P 47% | P 47% | P 47% | P 47% | أسباب الإباحة وموانع المسؤولية |

**Table 11.** Comparison with related Arabic-legal-RAG work. Metric column reports each paper's primary reported figure on its own benchmark.

| System | Domain | Primary metric | Score |
|---|---|---|---|
| El-Beltagy & Abdallah [1] | Arabic RAG (general) | embedding-model ablations | — |
| Hrimech et al. [2] | Moroccan family code | MRR, Recall@$k$, F1 | — |
| Aboasal et al. [5] | Egyptian legal IR | MAP / nDCG@10 | 0.83 / 0.86 |
| Abu Shairah et al. [3] | Saudi commercial law | verdict prediction | — |
| **Conan (this work)** | **Egyptian criminal law** | **hallucination rate** | **0.0%** |
| **Conan (this work)** | **Egyptian criminal law** | **validation pass rate** | **100%** |

- *Wider scope of source material*: 1,057 documents and 47,028 chunks vs. 500 articles / 1,000 Q&A in [5] and 2,500 Q&A pairs in [2].
- *Coverage of both substantive and procedural law* (Penal Code + Code of Criminal Procedure), with explicit prompt-level disambiguation. Prior Arabic-legal-RAG work tends to focus on one or the other.
- *Layered post-generation defence pipeline* not present in any of the cited prior work, which focus exclusively on the retrieval stage.

### *9.6. Cross-model benchmark (protocol)*

To situate Conan against general-purpose frontier models on Arabic legal questions, we provide a head-to-head harness (`benchmark_llms.py`) that queries our full RAG system alongside `openai/gpt-4o-mini` and `anthropic/claude-sonnet-4.5` (both routed through a single OpenRouter key for cost parity). It runs in two modes that isolate two distinct questions:

**Table 12.** Capability comparison with related Arabic-legal-RAG work. ✓ present, ✗ not reported. "Hybrid" = dense + sparse retrieval.

| Capability | El-Beltagy [1] | Hrimech [2] | Aboasal [5] | ALARB [3] | Conan |
|---|---|---|---|---|---|
| Hybrid retrieval (dense+sparse) | ✗ | ✗ | ✓ | ✗ | ✓ |
| Cross-encoder reranking | ✗ | ✗ | ✗ | ✗ | ✓ |
| Post-gen. hallucination defence | ✗ | ✗ | ✗ | ✗ | ✓ |
| Evidence/citation validation | ✗ | ✗ | ✗ | ✗ | ✓ |
| Agentic multi-agent pipeline | ✗ | ✗ | ✗ | ✗ | ✓ |
| Neuro-symbolic reasoning tool | ✗ | ✗ | ✗ | ✗ | ✓ |
| Both substantive + procedural law | ✗ | ✗ | ✗ | ✗ | ✓ |
| Document upload / ingestion | ✗ | ✗ | ✗ | ✗ | ✓ |
| Production service + wire contract | ✗ | ✗ | ✗ | ✗ | ✓ |
| Open released corpus | ✗ | ✗ | ✗ | ✓ | ✓ |

- *RAG mode* — all three models answer from the *same* retrieved context, isolating answer-synthesis quality from retrieval;
- *Raw mode* — each model answers from its own parametric knowledge with no retrieval, probing baseline Arabic criminal-law knowledge.

Each run emits a per-model CSV recording latency, answer length, and the set of cited article numbers, against which the same evidence-validation extractor (Section 6.3) is applied to measure citation grounding on equal footing. *Quantitative results are pending a full benchmark run and will be reported in a subsequent revision;* at the time of writing the comparison could not be executed because of API-access limits on the shared OpenRouter key. The protocol is documented here so the comparison is reproducible once access is restored.

### *9.7. Qualitative observations*

Two questions are particularly informative. **Q3** (التوقيف الاحتياطي): in v3, this triggered the wrong-law failure mode (cited Article 300 of the Penal Code); after prompt-level disambiguation in v4, the LLM correctly identifies the question as procedural and produces a refusal-style answer when retrieval does not surface the relevant chunks. Validation passes (no fabricated citation) at 59 % confidence. **Q18** (هل تعتبر الجريمة تامة أم شروع؟) was the most persistent failure: it failed in v3, regressed under v4's pure-prompt approach (citing Articles 30 and 31), and recovered to passing only with the article-lookup rescue in v6 ((تم استرجاع مواد إضافية من قاعدة البيانات للتحقق من الاستشهادات (مواد: 30, 31)), at 48 % confidence. These cases underline that no single mechanism is sufficient: each layer recovers a distinct class of failure.

### *9.8. Retrieval-quality optimisation (v9): embeddings, chunking, reranking, and a scored harness*

The v3→v8 evolution (Table 8) drove *citation fidelity*—what the system cites, given its context—to a 0 % hallucination rate. It did not address the complementary ceiling: whether the authoritative article ever *reaches* the context. A grounding pipeline can only validate citations against what retrieval surfaces; if the relevant article never enters the top-$k$, the best achievable outcome is an honest refusal, not a correct answer. The v9 work targets retrieval quality directly, along four axes, and introduces the measurement instrument that makes such tuning auditable.

*Motivation: the retrieval ceiling.* Three structural weaknesses bounded recall. (i) The embedder was `paraphrase-multilingual-MiniLM-L12-v2` (384-dim), a general-purpose multilingual encoder chosen for fast CPU rebuilds; it is not domain-tuned, is comparatively weak on Arabic legal phrasing, and its 512-token cap silently truncated longer statute chunks. (ii) Chunking was purely character-recursive, so a single article's text could be split across a boundary or share a chunk with an unrelated neighbour, diluting the signal for the exact unit a citation-driven system must retrieve. (iii) The cross-encoder reranker re-scored only the top-10 RRF candidates, so a relevant chunk fused at rank 11 or lower could never be promoted.

*Enhancement 1: embedding upgrade (MiniLM-384 → BGE-M3-1024).* We integrate `BAAI/bge-m3` (1,024-dim) behind the embedding configuration as the target encoder: a multilingual model with strong Arabic retrieval and native long-context support (sequence length raised 512 → 1,024, eliminating truncation), consistent with Aboasal et al. [5] reporting substantially higher MAP with BGE-M3 on a comparable Arabic legal task. As dimensionality changes, deploying it requires a full FAISS rebuild, performed into a staging directory and swapped atomically so the live index keeps serving. The rollout is deferred (see *Deployment status* below) but the path is fully prepared.

*Enhancement 2: article-aware chunking.* For statute-structured document types (Penal Code, Code of Criminal Procedure, legal-reference collections) the chunker splits on article-header boundaries (N المادة), emitting each article as a single intact chunk; only an article exceeding the doc-type chunk size is sub-split, and narrative material (cases, rulings, encyclopedia) keeps recursive character splitting. Each article-anchored chunk carries a new `primary_article` field—the article the chunk *is*, rather than one it merely cross-references—preferred when populating the per-source article field and feeding the article-validation and topic-match confidence signals. The chunker is the single source of truth shared by the batch builder and the incremental ingest pipeline, so the two index-writers cannot drift.

*Enhancement 3: wider reranker pool.* The cross-encoder now re-scores the top-50 RRF candidates rather than the top-10, rescuing relevant chunks fused at low rank. The cost is a modest per-query CPU increase and no rebuild—the one zero-cost, immediately-deployable lever of the four.

*Enhancement 4: a scored evaluation harness.* To make retrieval tuning measurable rather than impressionistic, `eval_harness.py` scores a labeled gold set and emits a machine-readable scorecard. A *retrieval mode* (LLM-free) measures the ceiling—recall@k, article-recall, primary-hit@k (whether a retrieved chunk *is* an expected article, a direct test of article-aware chunking), and MRR; an *end-to-end mode* scores citation recall, hallucination rate, out-of-scope abstention, confidence calibration, and latency against the live API. A `--compare` mode diffs two scorecards. This harness is the backbone for the before/after comparison once the embedding rollout lands, and for all future tuning.

*Deployment status and the CPU-embedding constraint.* The four enhancements differ in deployment cost. The widened reranker pool (Enhancement 3) is a runtime configuration change and is **live** with no rebuild. Article-aware chunking (Enhancement 2) is **implemented and validated**, and is applied online to every newly-ingested document; on a 100-file validation sample it produced 14,060 chunks of which **11,152 (79 %) were article-anchored**, confirming that the bulk of statute content is now cut at article boundaries rather than arbitrary character offsets. The scored harness (Enhancement 4) is **delivered**. The embedding upgrade (Enhancement 1), however, requires re-embedding the entire corpus, and here we report a concrete systems result: on the CPU-only reference machine (the GPU is unusable—its compute capability 5.0 is below the floor required by current PyTorch builds), BGE-M3 embeds at roughly **5.5 s per chunk** at 1,024-dim/1,024-token, which—compounded by the ∼3× chunk-count increase from article-aware splitting—projects to **multiple days** of continuous computation for a full-corpus rebuild. We therefore **defer the embedding rollout to a GPU-equipped environment** rather than peg the production CPU for days; the configuration, build/swap tooling, and evaluation harness are all in place so the rebuild and its before/after scorecard reduce to a single batch run plus a `--compare`. This is itself a useful deployment lesson for resource-constrained Arabic-RAG teams: the embedder that maximises retrieval quality may be infeasible to *build* without accelerator access, making the embedder choice a joint quality/operability decision.

*Positioning.* v9 is qualitatively different from v3–v8: the earlier layers are *defensive* (they prevent the model from asserting what it cannot support), whereas v9 is *generative of recall* (it raises what the model can legitimately support). The two are complementary—a higher ceiling converts more queries into grounded answers rather than honest refusals, without relaxing the 0 % hallucination property, since every citation still passes the same evidence validator.

## 10. Implementation, Deployment, and Integration

*Service and persona.* Conan is packaged as a FastAPI service with a thin router layer over the singleton retrieval, reranker, article-lookup, and session services, all loaded once at start-up. All user-facing output is in Modern Standard Arabic under a consistent government-style persona (“كونان”), enforced by the system prompts; the persona is deliberately formal and citation-bound rather than conversational.

*Reference UI.* A Streamlit application ships as the reference client, exercising every endpoint (Q&A, multi-turn chat with streaming, weakness analysis, defence memo, forensic check, summarisation, and document upload) and serving as living documentation of the wire contract.

*Containerisation and exposure.* The service is containerised with a `Dockerfile` and `docker-compose.yml` (documented in `DEPLOY.md`) for reproducible deployment. For demonstration and remote integration the laptop-hosted backend has been exposed over a Cloudflare quick tunnel, allowing an external frontend team to integrate against a live instance without dedicated hosting.

*Frontend integration contract.* The API is consumed by a separate .NET frontend. To support this, the response schema is treated as a frozen wire contract: the structured shape returned by `/qa`, `/chat`, `/weakness`, `/defense`, and `/forensic`—`answer` or `analysis` or `memorandum`, plus `confidence_score`, `confidence_factors`, `sources[]` (with per-source `legal_topic`, `article`, `referenced_articles`, `page`, `retrieval_score`, `rerank_score`), `warnings[]`, `conflicts_detected`, `latency_ms`, and `model`—is documented field-by-field in two contract files (`api_contract.md`, `backend_contract.md`) and field names cross the wire verbatim. Arabic-language clarification `warnings` are designed to be forwarded to the end-user as-is.

*Configurability.* Operational parameters are centralised in a single `config.py` (every value overridable by an environment variable), so behaviour can be tuned for a deployment without code changes; new env vars are mirrored in `.env.example`. Table 13 lists the most consequential tunables and their production defaults.

## 11. Strengths, Limitations, Challenges, and Lessons Learned

### *11.1. Strengths and practical impact*

The system's principal strengths follow directly from its design philosophy. *(1) Verifiable grounding*: every statutory citation is machine-checked against retrieved text, yielding a 0 % hallucinated-citation rate on the benchmark—the property that most directly governs whether a legal-AI tool can be trusted in practice. *(2) Defence in depth*: seven independently-toggleable grounding layers plus a six-agent case-analysis pipeline cover heterogeneous failure modes that no single mechanism addresses. *(3) Operability under constraint*: a five-provider failover chain, zero-token cache, rate gate, and cascade budget let the full system run on free LLM tiers and CPU-only hardware, which is the realistic setting for most academic and public-sector Arabic-legal deployments. *(4) An open corpus*: the released 1,057-document corpus lowers the entry barrier for future Egyptian-criminal-law work. *(5) Real integration*: a frozen wire contract and a live-tunnel deployment let an external .NET team build against the service, demonstrating the architecture beyond a notebook prototype.

### *11.2. Challenges encountered and how they were addressed*

Table 14 records the most significant engineering challenges and the mechanism each one motivated—making explicit the failure-driven nature of the development process.

**Table 13.** Principal configuration parameters and production defaults. All are environment-overridable through `config.py`.

| Parameter | Default | Effect |
|---|---|---|
| `RETRIEVAL_K` | 10 | Final chunks passed to the LLM |
| `RETRIEVAL_K_DENSE/SPARSE` | 60 / 60 | FAISS / BM25 candidate pool before fusion |
| `RETRIEVAL_K_RERANK` | 60 | Fused candidates re-scored by the cross-encoder |
| `RRF_K` | 60 | Reciprocal-Rank-Fusion constant |
| `RERANK_MAX_CONCURRENCY` | 2 | Cap on simultaneous CPU reranks |
| `CONFIDENCE_WEIGHTS` | .30/.20/.30/.20 | rerank / sources / article-valid. / topic-match |
| `RERANK_CALIBRATION_GAMMA` | 0.5 | Lifts squashed low sigmoid rerank scores |
| `ITERATIVE_K_SEQUENCE` | 10, 18 | Adaptive $k$-expansion on validation failure |
| `RETRY_MAX_ATTEMPTS` | 1 | Block-list corrective retries |
| `QA_CASCADE_BUDGET_S` | 90 | Wall-clock budget for the corrective cascade |
| `LLM_MIN_INTERVAL_S` | 2.0 | In-process rate gate between provider calls |
| `AGENT_MAX_WEAKNESSES` | 6 | Weaknesses given deep per-weakness research |
| `AGENT_RESEARCH_K` | 6 | Chunks retrieved per weakness |
| `MEMO_SELF_CHECK_MAX_ITERS` | 1 | Memo verify → revise passes |
| `MAX_CONTEXT_CHARS` / `MAX_INPUT_CHARS` | 12k / 50k | Context budget / max case-file size |
| `SESSION_MAX_TURNS` / `KEEP_RECENT` | 6 / 3 | Compaction trigger / verbatim window |
| `SESSION_TTL_HOURS` | 168 | Idle-session pruning (7 days) |

### *11.3. Lessons learned*

Four lessons generalise beyond this system. **(L1) Grounding is a post-generation problem as much as a retrieval problem**—improving the retriever alone never removed the parametric-memory citation failure; the defence layers did. **(L2) Move brittle reasoning out of the LLM when it can be computed**: the single largest case-analysis error (chronology) was eliminated not by a better prompt but by a few lines of deterministic code. **(L3) Not every clever idea helps**: heuristic synonym expansion measurably *regressed* retrieval (v5), a result we keep visible rather than bury. **(L4) The best model can be the wrong model**: on CPU-only hardware the highest-quality embedder is infeasible to *build*, making the embedder a joint quality/operability choice rather than a pure quality one.

### *11.4. Known limitations, weaknesses, and risks*

**Negative result on multi-query expansion.** In an intermediate version (v5) we evaluated heuristic synonym expansion (mapping التوقيف الاحتياطي to الحبس الاحتياطي etc.) as a query-side intervention. The result was a *regression*: pass rate dropped from 92.9 % (v4) to 78.6 % (v5), with four previously-passing questions re-introducing hallucinations. Inspection revealed that synonym expansion diluted retrieval quality by pulling in less-relevant chunks. This is consistent with the [1] observation that embedding-model

**Table 14.** Key challenges and the mechanisms they motivated.

| Challenge | Resolution |
|---|---|
| Legacy `.doc` / scanned-PDF corpus, unreadable to NLP tooling | Manual OCR + multi-stage Arabic cleaning of 933 files (Section 4) |
| LLM cites articles from parametric memory | Evidence validation + block-list retry + article-lookup rescue (Sections 6.3–6.5) |
| Relevant article never enters top-$k$ | Iterative $k$-expansion + widened rerank pool + article-aware chunking (Sections 6.6, 9.8) |
| Substantive-vs-procedural law confusion | Prompt-level law-naming disambiguation (Section 6.2) |
| Model inverts arrest-vs-warrant chronology | Deterministic, code-computed timing verdict injected as a hard fact (Section 7.2) |
| Free-tier token/minute exhaustion under multi-call requests | Rate gate, answer cache, single-retry cap, OpenRouter-primary (Section 5) |
| CPU rerank latency dominating per-query cost | Concurrency semaphore + cascade budget + bounded iterative-$k$ |
| Long memos hallucinate *arguments*, not just article numbers | Agentic draft → verify → revise self-check (Section 7.3) |
| BGE-M3 infeasible to build on CC-5.0 GPU | Defer rollout; prepare staging-build + atomic-swap tooling (Section 9.8) |

semantics for Arabic do not translate cleanly across closely-related lexically-distinct legal terms. We retain the implementation (`USE_MULTI_QUERY=False` by default) and report this as a cautionary data point. A semantically-aware query expansion using the LLM itself, rather than a static synonym table, may recover this lost potential.

**Retrieval ceiling (the dominant residual risk).** The grounding pipeline can only validate citations against what retrieval surfaces; if the authoritative article never reaches the context, the best outcome is an honest refusal, not a correct answer. With the live MiniLM-384 index this ceiling is the single largest bound on answer correctness, and it is exactly what the deferred BGE-M3 upgrade targets (Section 9.8).

**LLM cost and latency.** The full pipeline can incur up to three LLM calls per hard Q&A and one call per agent for case work. The OpenRouter-primary move removed the token-per-minute ceiling, but CPU rerank latency still makes a full agentic memo a multi-minute operation—acceptable for considered legal drafting, but a bottleneck for interactive use until a GPU retrieval backend is available.

**Article-lookup precision.** The rescue mechanism operates on chunk *references* to article numbers, not on chunks that *define* those articles; in ∼5 % of invocations the surfaced text mentions the article in passing rather than stating it. This suffices for validation but is suboptimal for answer quality.

**Evaluation scope.** The 28-question benchmark is small relative to the 13,000-case ALARB [3] and the 2,500-pair set of [2]; it was sized for fast, repeatable, manually-

auditable iteration. The cross-model benchmark against frontier LLMs (Section 9.6) is implemented but its quantitative run is still pending API access.

## 12. Future Work

We group planned work into five themes, ordered roughly by expected impact on answer correctness.

*Retrieval ceiling: complete the BGE-M3 rollout.* The highest-leverage next step is the embedding upgrade already prepared behind configuration (Section 9.8): re-embedding the full corpus with BGE-M3 (1,024-d, 1,024-token, eliminating MiniLM's 512-token truncation) on GPU-equipped hardware, then running the scored harness's `--compare` to quantify the recall gain. Article-aware chunking and the widened rerank pool are already live; the embedder is the last and largest deferred lever, blocked only by the CC-5.0 GPU constraint, not by missing engineering. A dedicated *article-definition* index—paragraph-level segmentation of the Penal Code so the rescue path returns the article's *text* rather than a passing cross-reference—would further raise rescue answer quality.

*Toward GraphRAG and a knowledge graph.* The current system is the Phase-1 instantiation of a longer roadmap. The natural architectural evolution is a legal knowledge graph linking articles, offences, defences, and precedents, over which GraphRAG-style multi-hop retrieval could answer questions that require chaining several articles (e.g. aggravating-circumstance + base-offence + penalty) rather than retrieving each in isolation. The `primary_article` / `referenced_articles` metadata already extracted is the seed for such a graph.

*Deeper and more autonomous agents.* The six-agent pipeline is a fixed DAG; future versions could let the orchestrator decide *dynamically* how many weaknesses to research and how deep to retrieve, add a dedicated *cross-examination* agent that adversarially attacks each drafted argument, and add tool-use agents (statute-version lookup, deadline calculators) alongside the existing deterministic timing tool. A semantically-aware, LLM-driven query expansion—the principled successor to the synonym-table approach that regressed in v5—belongs here too.

*Evaluation at scale.* The 28-question benchmark was sized for fast, auditable iteration. Scaling to a labelled gold set of comparable size to ALARB [3] (using the released corpus), completing the pending cross-model benchmark against frontier LLMs (Section 9.6), and adding human expert-rated reasoning-quality scoring would strengthen the camera-ready evaluation. Confidence-calibration curves over the larger set would also let the clarification threshold be set empirically rather than heuristically.

*Scalability, robustness, and a possible rebuild.* For higher load the singleton-on-one-process design would move to a horizontally-scaled deployment: the read-only FAISS/BM25 indices replicated behind a load balancer, sessions and the answer cache backed by a shared store (Redis) rather than per-process JSON, and the CPU reranker moved to a GPU inference service to remove the dominant latency cost. A future major version could also be *rebuilt* natively around the knowledge graph and an agent framework (e.g.

a typed tool/agent runtime) rather than retrofitting agents onto the RAG core—trading the current minimal, dependency-light design for richer orchestration once the quality ceiling of the retrieval upgrade has been realised. Security and compliance hardening (authentication, rate limiting, audit logging, and PII handling for uploaded case files) is a prerequisite for any real-world legal deployment.

## 13. Conclusion

We have presented three complementary contributions: an open Arabic criminal-law corpus, manually constructed from 933 legacy-format source files into 1,057 cleaned documents and released publicly on HuggingFace; **Conan**, a hallucination-resistant Arabic legal RAG system built on top of this corpus, whose seven-layer grounding-defence pipeline drives the rate of hallucinated statutory citations from a baseline of 28.6 % down to **0.0 %** on a 28-question benchmark; and a six-stage multi-agent agentic pipeline that brings the same grounding discipline to open-ended case work, anchoring every argument in retrieved authority and replacing a stubborn chronological-reasoning failure with a deterministic, code-computed verdict.

Our key methodological contribution is the demonstration that **no single mechanism is sufficient** for hallucination suppression in Arabic legal RAG. The baseline failures distribute across distinct root causes (LLM memory citations, retrieval coverage gaps, fragmentary user inputs, wrong-law confusion, prompt-template leakage), and each requires its own layer of defence; open-ended legal reasoning additionally benefits from decomposition into specialised, individually-grounded agents and from moving brittle reasoning out of the model when it can be computed. The system's practical impact is to show that a trustworthy, verifiable Arabic legal assistant can be built and operated under genuine resource constraints—free LLM tiers and CPU-only hardware—which is the realistic setting for most academic and public-sector deployments. We hope the released corpus, the layered defence pipeline, and the agentic case-analysis design inform and accelerate the deployment of Arabic legal-AI systems in production settings.

## Acknowledgements

The authors thank the open-source community behind FAISS, BM25, BGE-Reranker-v2-M3, Groq, Google Gemini, and OpenRouter, and the Egyptian legal-publication archives whose collections were the raw material for the released corpus.

## References

[1] S. R. El-Beltagy and M. A. Abdallah, "Exploring Retrieval Augmented Generation in Arabic," *Procedia Computer Science*, vol. 244, pp. 296–307, 2024. 6th International Conference on AI in Computational Linguistics (ACLing 2024). doi:10.1016/j.procs.2024.10.203.

[2] J. Hrimech, M. Mghari, and Y. Zaz, "Retrieval-augmented generation for Arabic legal information: the family code case study," *TELKOMNIKA Telecommunication Computing Electronics and Control*, vol. 23, no. 6, pp. 1495–1505, Dec. 2025. doi:10.12928/TELKOMNIKA.v23i6.27400.

[3] H. Abu Shairah, S. AlHarbi, A. AlHussein, S. Alsabea, O. Shaqaqi, H. AlShamlan, O. Knio, and G. Turkiyyah, "ALARB: An Arabic Legal Argument Reasoning Benchmark," in *Proceedings of the Third Arabic Natural Language Processing Conference*, pp. 389–406, Association for Computational Linguistics, Nov. 8–9, 2025.

[4] M. Alghamdi, M. Abushawarib, M. Ellouh, M. Ghaleb, and M. Felemban, "Enhancing Arabic Information Retrieval for Question Answering," in *ICFNDS '23: Proceedings of the 7th International Conference on Future Networks and Distributed Systems*, Dubai, UAE, Dec. 21–22, 2023. doi:10.1145/3644713.3644763.

[5] R. Aboasal, S. Montasser, F. Hossam Eldin, A. Abdelwahab, and H. Abdelazim, "Arabic Legal Information Retrieval: The Impact of Morphological Segmentation and Semantic Embeddings," *Procedia Computer Science*, vol. 275, pp. 275–282, 2026. 7th International Conference on AI in Computational Linguistics (ACLing 2025).

[6] "Mini-RAG Project: Learnings & Technologies," internal RAG-implementation documentation, on file with the authors.

[7] M. Nageh, "Egyptian_Criminal_Legal_Assistant_RAG_V2 [Dataset]," HuggingFace Datasets, 2026. Available: https://huggingface.co/datasets/Mirna2Nageh/Egyptian_Criminal_Legal_Assistant_RAG_V2.